

Practical LAMBDA Programming

Building Real Systems in Excel & Google Sheets

Part 2 of 2 · Applications & Practice

Anonymous · April 2026

A LAMBDA Is Just a Function Definition

Mathematical Form

```
f(x) = x + 1  
f(2) = 3
```

Spreadsheet Form

```
=LAMBDA(x, x+1)  
=LAMBDA(x, x+1)(2) → 3
```

General Form

```
=LAMBDA( param1, param2, ... , function_expression )  
  
f(param1, param2, ...) = function_expression (param1, parma2, ... )
```

KEY IDEA

A LAMBDA is just a function definition — parameters plus a function expression.

Functions with Multiple Parameters

Mathematical Form

$$\begin{aligned} f(x, y) &= x + y & f(2, 3) &= 5 \\ g(x, y, z) &= x + y + z & g(1, 2, 3) &= 6 \end{aligned}$$

Calling a function

$$f(x) = (x+1); \quad f(5) \rightarrow 6$$

Spreadsheet Form

$$\begin{aligned} &= \text{LAMBDA}(x, y, x + y)(2, 3) && \rightarrow 5 \\ &= \text{LAMBDA}(x, y, z, x + y + z)(1, 2, 3) && \rightarrow 6 \end{aligned}$$

Calling a LAMBDA

$$\begin{aligned} &= \text{LAMBDA}(x, x+1)(5) && \rightarrow 6 \\ &\text{// Define and execute immediately} \end{aligned}$$

KEY IDEA

A LAMBDA consists of parameters + a function expression, just like a regular function definition.

Named Functions

Excel

Name Manager
(Formulas → Define Name)

Google Sheets

Named Functions
(Data → Named Functions)

- Define once
- Reuse anywhere in the workbook
- Share across sheets

Introducing LET

LET returns a function expression. It allows you to break a LAMBDA's function expression into named sub-expressions.

Spreadsheet

```
=LET(  
  x,      5,  
  result, x + 5,  
  result  
)
```

Equivalent Code

```
{  
  x      = 5;  
  result = x + 5;  
  result;  
}
```

KEY IDEA

LET lets you break an expression, into sub-expressions, and name the sub-expressions. The value of the LET is the last sub-expression. IN standard-code, Let is equivalent to a code block delimited by curly braces.

BEST PRACTICE

The "result" Pattern

Always make the last sub-expression in a LET, be a single variable name (example “result”).

```
=LET(  
  x,      5,  
  y,      10,  
  result, x + y,  
  result           // ← always return result  
)
```

Why this matters

- Single inspection point
- Easy to isolate errors
- Step-by-step validation
- Prevents nested formula complexity
- Acts like a return variable

MAP — Apply a Function to Every Element

```
=MAP(A1:A10,  
    LAMBDA(x,  
        LET(  
            result, x * 2,  
            result  
        )  
    )  
)
```

Input

Accepts an array of values

Apply

Applies a LAMBDA to each element

Output

Returns an array of the same size

KEY IDEA

MAP transforms every element of an array without loops.

REDUCE — Aggregate an Array to a Single Value

```
=REDUCE(0, A1:A10,  
  LAMBDA(acc, x,  
    LET(  
      result, acc + x,  
      result  
    )  
  )  
)
```

Initial Value

Starts with an accumulator (here: 0)

Iterate

Applies LAMBDA sequentially to each element

Accumulate

Updates acc at each step

Output

Returns a single final accumulated value

KEY IDEA

REDUCE combines an array into a single result through iterative accumulation.

SCAN — Running Accumulation

```
=SCAN(0, A1:A10,  
  LAMBDA(acc, x,  
    LET(  
      result, acc + x,  
      result  
    )  
  )  
)
```

SCAN vs REDUCE

	REDUCE	SCAN
Input	array	array
Output	1 value	array (same size)
Shows	final acc	all intermediate values
Use for	totals	running totals / debugging

KEY IDEA

SCAN is like REDUCE, but returns all intermediate values — ideal for running totals and debugging.

ADVANCED

Higher-Order Functions

```
=MAP(range, LAMBDA(x, f(x)))
```

Functions are first-class objects:

Passed as inputs

A LAMBDA can accept another LAMBDA as a parameter

Returned as outputs

A LAMBDA can return another LAMBDA

Composed

Functions can be chained to build complex behavior

KEY IDEA

This is real programming — LAMBDA enables functional composition inside a spreadsheet.

ADVANCED

Recursion

A recursive function calls itself. Example: `sumn(x)` — sums integers from `n` down to 1.

```
=LAMBDA(x,  
  IF(  
    x = 1,  
    1,           // base case  
    x + sumn(x - 1) // recursive step  
  )  
)
```

How it works

- Base case: $x = 1 \rightarrow \text{return } 1$
- Recursive step: `return x + sumn(x-1)`
- Reduces the problem at each step
- Must be saved as a named function to self-reference

KEY IDEA

Solve a problem by reducing it to a smaller version of itself.

The Function Library Pattern

Managing LAMBDA's scattered across Name Manager is hard. The solution: a centralised FUNCTION_LIB.



Library = source of truth · Workbook = execution environment

- Centralised control
- Transparent design
- Version control
- Reusable across workbooks
- Easier debugging
- Portability

The Functional Dataflow Model



- No explicit loops — iteration is expressed through array functions
- No mutation — values are transformed, not overwritten
- Computation expressed as pure transformations of data
- Programs are built by composing functions

Additional Functions

Professional applications use many functions beyond the introductory set — but not all are common to both platforms.

COMMON TO BOTH EXCEL & GOOGLE SHEETS

BYROW / BYCOL MAKEARRAY FILTER CHOOSECOLS / CHOOSEROWS
HSTACK / VSTACK TOCOL / TOROW WRAPROWS / WRAPCOLS

EXCEL ONLY

TAKE / DROP
EXPAND
ISOMITTED

GOOGLE SHEETS ONLY

QUERY
ARRAYFORMULA
FLATTEN

KEY IDEA

The LAMBDA helpers — MAP, REDUCE, SCAN, BYROW, BYCOL, MAKEARRAY — work in both platforms; always confirm support before relying on newer array-shaping functions.

Common Design Patterns

As applications grow, reusable design patterns become increasingly important for keeping systems maintainable.

STRUCTURE & REUSE

- Function libraries
- Function composition
- Higher-order functions
- Recursive algorithms
- Optional parameters

DATA TRANSFORMATION

- Data pipelines
- Row-wise & column-wise processing
- Array generation
- Table transformation
- Error-handling wrappers

KEY IDEA

Reusable patterns — libraries, composition, and data pipelines — keep large spreadsheet systems readable and maintainable as they grow.

Key Takeaway

LAMBDA turns the spreadsheet into a functional programming environment.

- Functions are the primary building blocks
- Iteration expressed without loops
- Complex formulas decomposed into small named sub-expressions
- Define once, reuse everywhere
- LAMBDA is a complete programming model — any computable function can be expressed using lambda calculus

Spreadsheets are evolving into programmable environments.

LAMBDA provides:

Structure

Abstraction

Scalability

April 2026